

NAME:KRISHNAPRIYA P

REG NO:192511279

Course code:CSA0613

Course title: Design and Analysis of Algorithms

CO1_ASSESSMENT TOOL 1 _Application-Based Question

1 .Title :Application of the Substitution Method for a Recursive Financial Summation Engine

Aim

To solve the recurrence relation

$$T(n) = T(n/2) + n$$

Software Used

- Python

Algorithm

1. Read transaction values.
2. Divide the list into two halves.
3. Recursively compute the sum.
4. Combine the results.
5. Display the total summary.

Source Code (Python)

```
main.py    Share  Run
1 def financial_sum(arr):
2     if len(arr) == 1:
3         return arr[0]
4
5     mid = len(arr) // 2
6     return financial_sum(arr[:mid]) + financial_sum(arr[mid:])
7
8 transactions = [100, 250, 300, 150, 400, 500, 350, 450]
9 print("Total =", financial_sum(transactions))
```

OUTPUT

```
Output
Total = 2500
=== Code Execution Successful ===
```

Analysis

Recurrence

$$T(n) = T(n/2) + n$$

Expansion

$$\begin{aligned} T(n) &= T(n/2) + n \\ &= T(n/4) + \frac{n}{2} + n \\ &= T(n/8) + \frac{n}{4} + \frac{n}{2} + n \end{aligned}$$

After $k = \log n$ expansions,

$$T(n) = T(1) + 2n$$

Therefore,

$$\boxed{T(n) = \Theta(n)}$$

Verification

Assume $T(n) = cn$.

$$\begin{aligned} cn &= c\left(\frac{n}{2}\right) + n \\ c &= 2 \end{aligned}$$

Hence,

$$T(n) = \Theta(n)$$

Documentation

- The algorithm recursively divides the transaction list into halves.
- Linear work is performed at each recursive level.
- Suitable for processing large financial transaction datasets efficiently.
- **Time Complexity: $\Theta(n)$**
- **Space Complexity: $\Theta(\log n)$**

2.Title : Case-Based Analysis of Merge Sort Algorithm Efficiency

Aim

To analyze the efficiency of Merge Sort under best, average, and worst-case scenarios using asymptotic notations..

Software Used

- Python

Algorithm

- Divide the array into two equal halves.
- Recursively sort the left half.
- Recursively sort the right half.
- Merge the two sorted halves.
- Repeat until the entire array is sorted.

Source Code

```
main.py  [ ] [ ] [ ] Share Run
1 ~ def merge_sort(arr):
2 ~     if len(arr) > 1:
3 ~         mid = len(arr) // 2
4 ~         left = arr[:mid]
5 ~         right = arr[mid:]
6 ~
7 ~         merge_sort(left)
8 ~         merge_sort(right)
9 ~
10 ~     i = j = k = 0
11 ~
12 ~     while i < len(left) and j < len(right):
13 ~         if left[i] < right[j]:
14 ~             arr[k] = left[i]
15 ~             i += 1
```

OUTPUT

```
Output Clear  
Sorted Array: [3, 9, 10, 27, 38, 43, 82]  
  
=== Code Execution Successful ===
```

Analysis

Merge Sort uses the divide-and-conquer approach by splitting the array into halves and merging them after sorting.

- Best Case: Already sorted input $\rightarrow \Theta(n \log n)$
- Average Case: Random input $\rightarrow \Theta(n \log n)$
- Worst Case: Reverse sorted input $\rightarrow \Theta(n \log n)$

Since Merge Sort performs the same operations in all cases, its running time remains $\Theta(n \log n)$. It requires $\Theta(n)$ extra space for merging and is efficient for large datasets.

Documentation

- Merge Sort recursively divides the input array into two halves.
- The sorted halves are merged to produce the final sorted array.
- The algorithm performs consistently in the best, average, and worst cases.
- It guarantees efficient sorting for large datasets.
- **Time Complexity:** $\Theta(n \log n)$ (Best, Average, and Worst Case)
- **Space Complexity:** $\Theta(n)$ (Auxiliary space required for merging)